

ADC

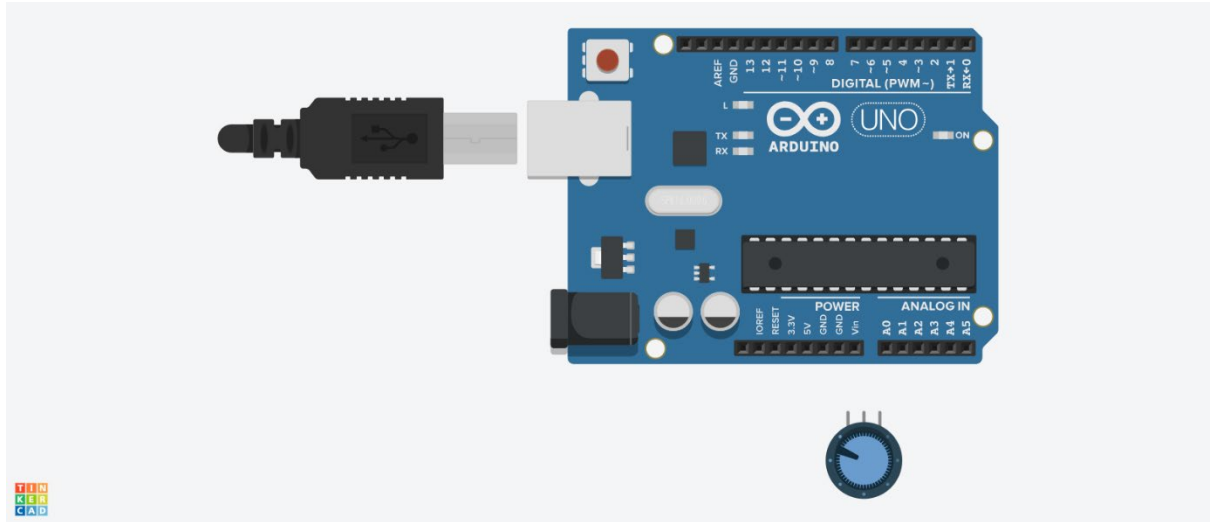
TinkerCAD Activity: "A2D"

This practice includes the following topics:

- ADC Configuration.
- ADC Interrupts.
- ADC & Timers.
- Potentiometer Sensor.
- Photoresistor Sensor.

1. Tinker: "1. Introduction to Analog Configuration"

- a. Go to "Activities" and "Copy and Tinker" the associated task.
- b. Update your "submission file name".



- c. Write a code that does the following:
 - i. Configuring the ADC based on the code template and its guidelines.
 - ii. Exploring the Potentiometer: Reading the digital values after the ADC (using "serial") that the Potentiometer provides.Testing it out using simulation
- d. Objective: Create a simple Arduino-based system that reads an analog input (from a potentiometer), converts it to a digital value using the built-in Analog-to-Digital Converter (ADC), and then prints this value to the Serial Monitor every 400 milliseconds.
- e. Guidelines:
 - i. Setup the hardware: Connect your analog device (potentiometer) to the ADC0 channel of your Arduino board.
 - ii. Configure ADC: Set up the ADC to use AVcc as the reference voltage and select ADC4 as the input channel. Use a prescaler of 128 to ensure that the ADC operates within its recommended frequency.
 - iii. Read ADC Value: In the main loop, start an ADC conversion with each iteration. Wait for the conversion to complete by checking the ADSC bit in ADCSRA.
 - iv. Process and Display ADC Value: Once the conversion is complete, read the ADC value, which is the digital representation of your analog input and display it.
 - v. Delay: Add a delay of 400 milliseconds between each reading to make the output more readable and to prevent flooding the Serial Monitor with too much data too quickly.

2. Tinker: “2. Introduction to Analog Configuration - using Interrupts”

- a. Go to “Activities” and “Copy and Tinker” the associated task.
- b. Update your “submission file name”.
- c. Objective: After completing the previous exercise, modify your system to use an interrupt-driven approach for reading and processing the analog input. This will allow your program to respond to ADC conversions more efficiently and free up the main loop for other tasks.
- d. Guidelines:
 - i. Hardware Setup should be as in exercise (1). You can simply copy-paste the Hardware from this exercise.
 - ii. Enable ADC interrupt: In the setup function, enable ADC interrupts by setting the ADIE bit in the ADCSRA register. This will allow your program to respond to ADC conversion complete events using an interrupt service routine (ISR).
 - iii. ADC Interrupt Service Routine: Create an ISR for the ADC_vect interrupt vector. This function will be called automatically when an ADC conversion is completed. Inside the ISR, read the ADC value, and if it's above 400 only then print it to the Serial Monitor, and start the next ADC conversion.
 - iv. Adapt your code: Make necessary modifications to implement the interrupt-driven approach as described above. You should already have the code for setting up the ADC and reading the ADC value. Focus on the changes needed to enable ADC interrupts, create the ISR, and move the ADC processing to the ISR (from the main loop).
- e. NOTE: Make sure you (1) completed successfully previous exercise, and (2) you covered the lessons and video practice regarding interrupts & ADC on Moodle.

3. Tinker: “3. ADC & Timers”

- a. Go to “Activities” and “Copy and Tinker” the associated task.
- b. Update your “submission file name”.
- c. Objective: After completing the concept of interrupt-driven programming with an emphasis on ADC conversions, it’s time to integrate a timer interrupt. Specifically, your goal is to modify your ADC conversions so that they are initiated every 4 seconds, rather than continuously.

d. Guidelines:

- 1. Hardware Setup should be as in exercise (1). You can simply copy-paste the Hardware from this exercise.

- 2. Setup Timer1: In the setup function, configure Timer1 to generate an interrupt every second.

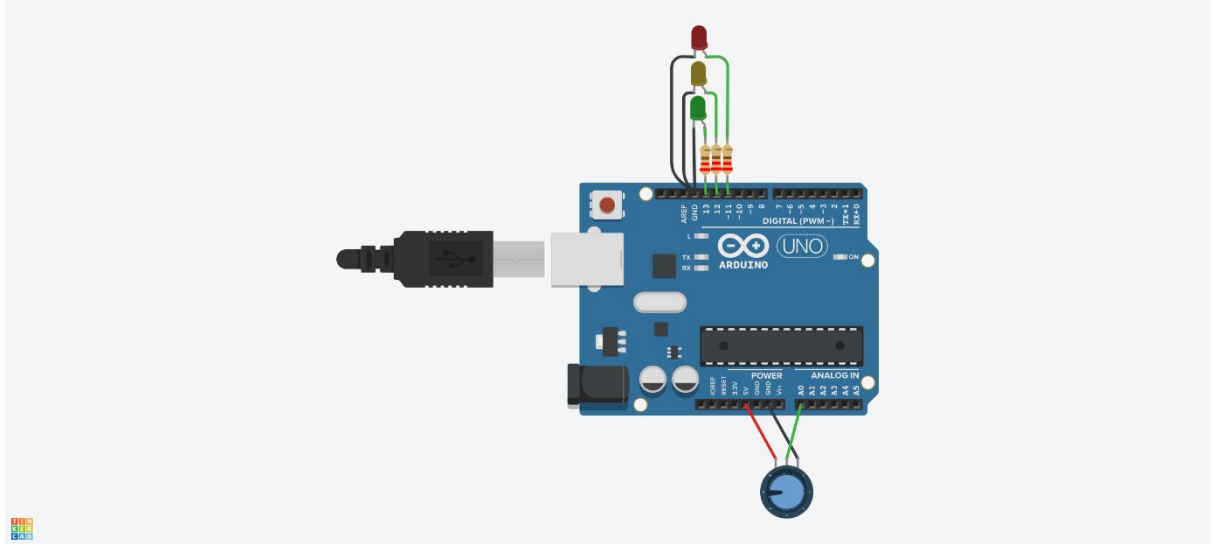
This involves setting up the Timer1 control registers (TCCR1A and TCCR1B), the Timer1 counter register (TCNT1), the Timer1 output compare register A (OCR1A), and the Timer1 interrupt mask register (TIMSK1).

- 3. Timer1 Interrupt Service Routine: Create an ISR for the TIMER1_COMPA_vect interrupt vector.
- 4. Modify ADC ISR: Update your ADC_vect ISR to only read the ADC value and print it to the Serial Monitor. The initiation of the next ADC conversion should now be handled by the Timer1 ISR.

4. Tinker: "4. Potentiometer with LEDS"

(Traffic Light Simulation Using ADC and Interrupts)

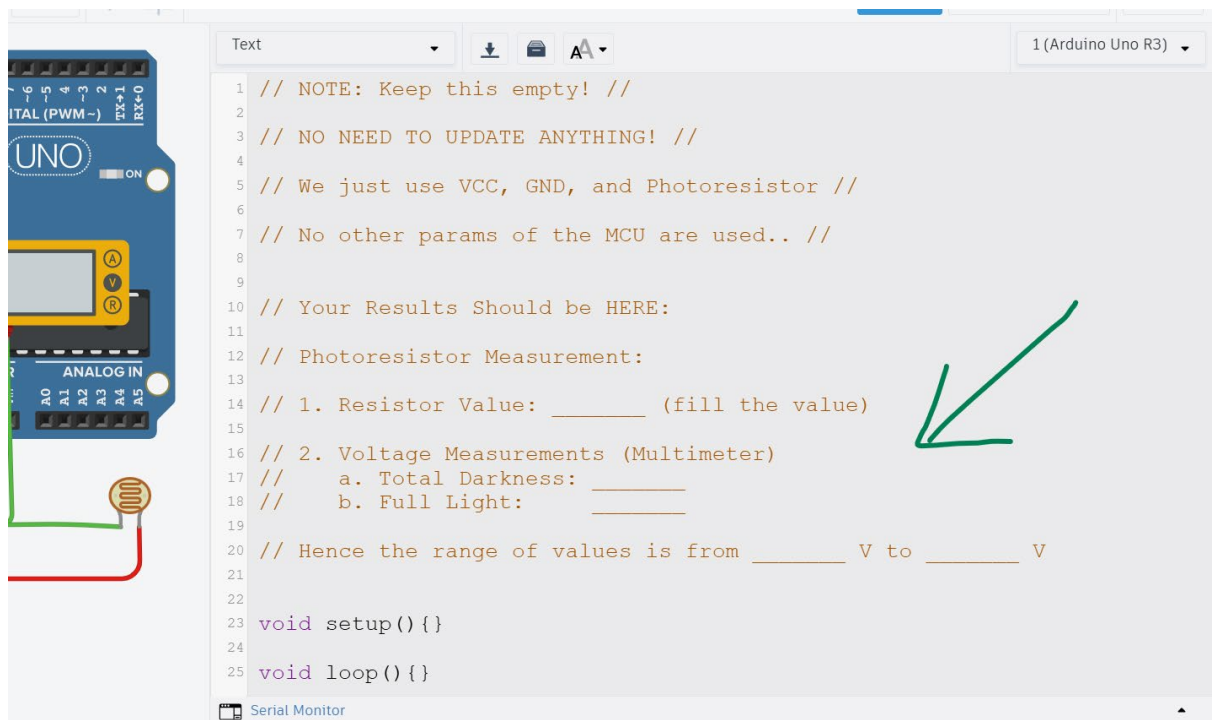
- a. Go to "Activities" and "Copy and Tinker" the associated task.
- b. Update your "submission file name".



- c. Objective: Create an Arduino-based traffic light system that uses three LEDs to mimic the behavior of a traffic light. The light that is on will be determined by the value obtained from an Analog-to-Digital Converter (ADC) reading of a Potentiometer.
- d. Guidelines:
 - i. Setup the hardware: Connect three LEDs to digital pins 11 (PB3), 12 (PB4), and 13 (PB5) on your Arduino board, which will represent the Red, Yellow, and Green lights respectively.
 - ii. Configure ADC: Set up the ADC to use AVcc as the reference voltage and select ADC0 as the input channel. Use a prescaler of 128 to ensure that the ADC operates within its recommended frequency.
 - iii. The ADC reading will be divided into three ranges:
 1. The first part (0-341) will correspond to the green light.
 2. The second part (342-682) will correspond to the yellow light.
 3. The third part (683-1023) will correspond to the red light.
 - iv. Use Interrupts: Implement an ADC Interrupt Service Routine (ISR) to read the ADC value when the conversion is complete. Based on the ADC value, decide which LED should be on, and start the next ADC conversion.

5. Tinker: “5. Photoresistor Measurement using Multimeter”

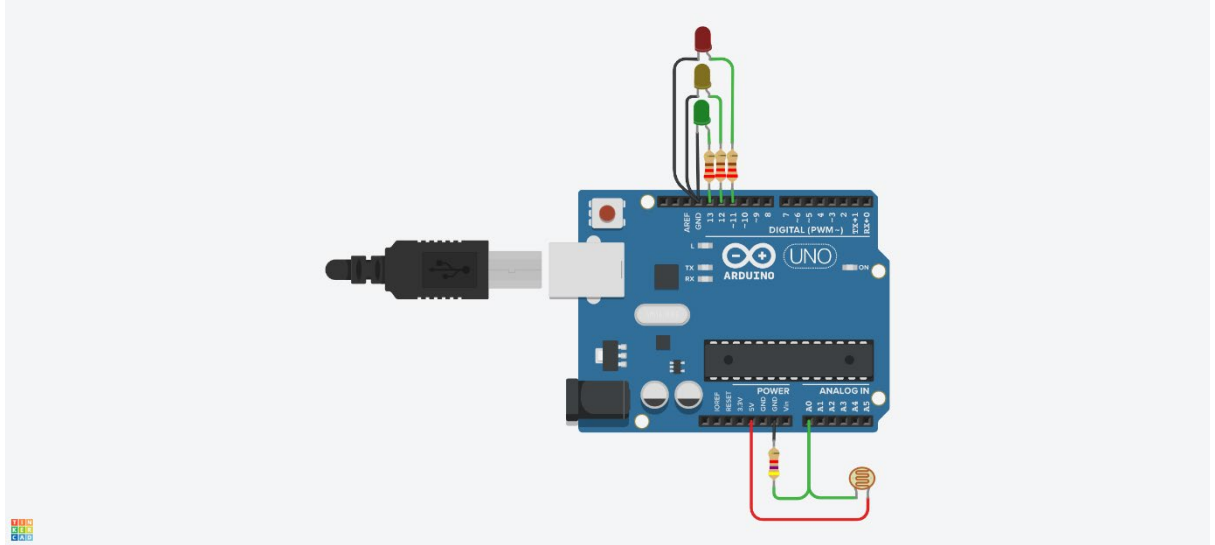
- a. Go to “Activities” and “Copy and Tinker” the associated task.
- b. Update your “submission file name”.
- c. Before writing code, you must understand your hardware. A photoresistor’s resistance changes with light, but the Arduino only understands **Voltage**. We use a **Fixed Resistor** to turn those resistance changes into a Voltage range we can measure.
- d. Task:
 - Start the Simulation.
 - Fill in the blanks based on your Multimeter readings.
 - a. Observe the value of the Resistor (the Resistor, not Photoresistor!).
 - b. Click on the Photoresistor to slide the light level from Dark to Bright
 - i. Observe and fill the Voltage on the Multimeter at “Total Darkness”.
 - ii. Observe and fill the Voltage on the Multimeter at “Full Light”.
 - Submit the file as any other file requirements (“.ino”).
 - NOTE: Don’t change anything in the HARDWARE!!



6. Tinker: “6. ADC & Photoresistor”

(Arduino Light Indicator with Photoresistor)

- a. Go to “Activities” and “Copy and Tinker” the associated task.
- b. Update your “submission file name”.



- c. Objective: The goal of this exercise is to create a program for an Arduino UNO R3 that will control a set of LEDs based on the amount of light detected by a photoresistor. The setup involves one photoresistor and three LEDs (red, yellow, and green) connected to the Arduino.
- d. Guidelines:
 - i. Use the ADC of the Arduino to read the voltage across the photoresistor continuously, which is inversely proportional to the amount of light incident on the photoresistor. The ADC should be set up to use AVcc as the reference voltage and a prescaler of 128.
 - ii. Divide the ADC range (0-1023) into three parts.
 1. The first part (0-341) represents low light.
 2. The second part (342-682) represents medium light.
 3. The third part (683-1023) represents high light.
 - iii. In response to the ADC readings, control the state of the LEDs as follows:
 1. If the light level is low (first part), turn on only the green LED.
 2. If the light level is medium (second part), turn on both the green and yellow LEDs.
 3. If the light level is high (third part), turn on all the LEDs (red, yellow, and green).
 - iv. Print the ADC value to the Serial Monitor after each ADC conversion.